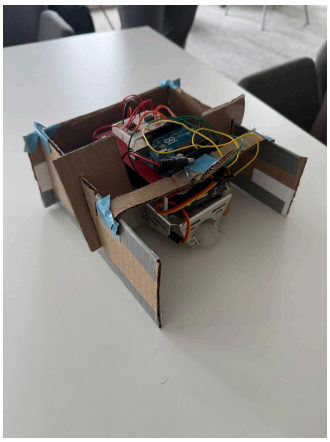
Mechatronics Robot Competition Report

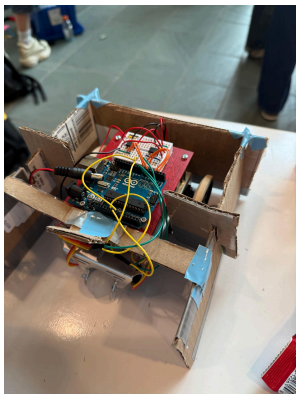
Group 57: Dylan Mies - djm494, Olivia Tolliver - ot64, Benjamin Okoronkwo - bco26

Robot Design and Strategy Overview



Our robot for this year's Cube Craze Mechatronics competition consists of a cardboard square enclosure, two QTI sensors to detect the field border, and a pre-programmed path to maximize block collection (see Appendix D).

The square enclosure fits within the 8x8 inch starting constraint, with an open front to allow blocks to enter and gaps between the wheels and walls so blocks slide in and stay trapped as the robot drives over them (see Appendix C). The robot is controlled by an Arduino Uno, which drives two motors through a pair of L9110 H-bridge motor drivers powered by a 9V battery, with the QTI sensors feeding line detection data back to trigger turns (see Appendix B).



The robot starts at roughly a 45 degree angle against the black border, drives forward until a QTI sensor detects the border, then makes a 135 degree turn to face 180 degrees. It drives straight down the middle of the field to collect as many blocks as possible in a single pass, then continues in an outward spiral, turning left each time it detects the border and increasing its drive distance with each pass until the match ends.

Design Process Reflection

Our primary goals for this competition were to collect as many blocks as possible, ensure they remained within our robot's perimeter, keep the robot within the 8x8 inch starting constraint, and execute a path that would maximize block collection. We also had to work within strict budget and time constraints that significantly limited what we could actually build.

Originally, we wanted to create a custom chassis with two spinning intakes to capture and retain blocks, along with two arms that would deploy in a V-shape to funnel blocks inward. The chassis would store blocks underneath, surrounded by a square enclosure with an open front, with the intakes mounted inside between the chassis and the outer walls. This was not feasible given our constraints, so we stripped the design down to just the box enclosure around the chassis, with gaps between the wheels and the walls for blocks to slide in.

We built a cardboard prototype for Milestone 4, which worked well enough that we formalized it into a CAD drawing (see Appendix C) and submitted it to the RPL for laser cutting in acrylic. The laser cutter malfunctioned and only cut partway through the acrylic, and with the competition approaching we had no time to resubmit. We ended up building an upgraded cardboard version of the design instead, reflected in the Bill of Materials in Appendix A.

Competition Analysis

Our robot performed better than expected. We won the first two matches and attribute that to the build of the robot. Although the shell was made of cardboard, the back was very robust and because of it, we were able to not be pushed off the field and hold our own during interference.

By the third match, we started to see our robot veer off the field. Since we switched fields, we believe this may have been caused by inconsistent QTI sensor readings. The sensors may have detected the border differently depending on the field surface, lighting conditions, and time of day. We also believe reduced battery power over time may have made the motion and control weaker, especially when the robot was pushed by opponents.

One thing that was unexpected was this robot we coined, “The Wall”, that would shoot this wall mechanism toward our bot and block us from getting blocks. Since their wall spanned the whole field, we weren’t able to get past them.

Our robot’s strengths lie in our structure and with creating a body that is stiff and able to hold its own. Our weakness lies in our sensing abilities and fine tuning our QTI Sensors over different tracks and lighting which could improve with more time and testing.

Conclusions

Overall, our group did a good job on this project, but there was definitely some room for improvement. If we were to do this project again, it would likely be beneficial to make changes to our strategy (what the robot is coded to do), the electrical components we used, and the physical materials we used for the outer portion of the robot.

When we first started the project, we decided to code the robot to follow a set path for each match. Looking back at how the robot performed, it was not a good choice to assume that the robot would be able to follow this path. In almost every match, our bot ended up colliding with the other robot, forcing our bot off its path and often off the board as well. If we had another attempt to fix this issue, it would likely be a good idea to focus more on designing code that keeps the robot on the board.

Continuing with the problem of staying on the board, we could add a color sensor onto the robot instead of just relying on the QTI sensors. Using a color sensor to differentiate between yellow and blue, and using the QTI sensors to detect when the robot hits the black border would be a great way to mitigate the color sensing issues we saw during the competition.

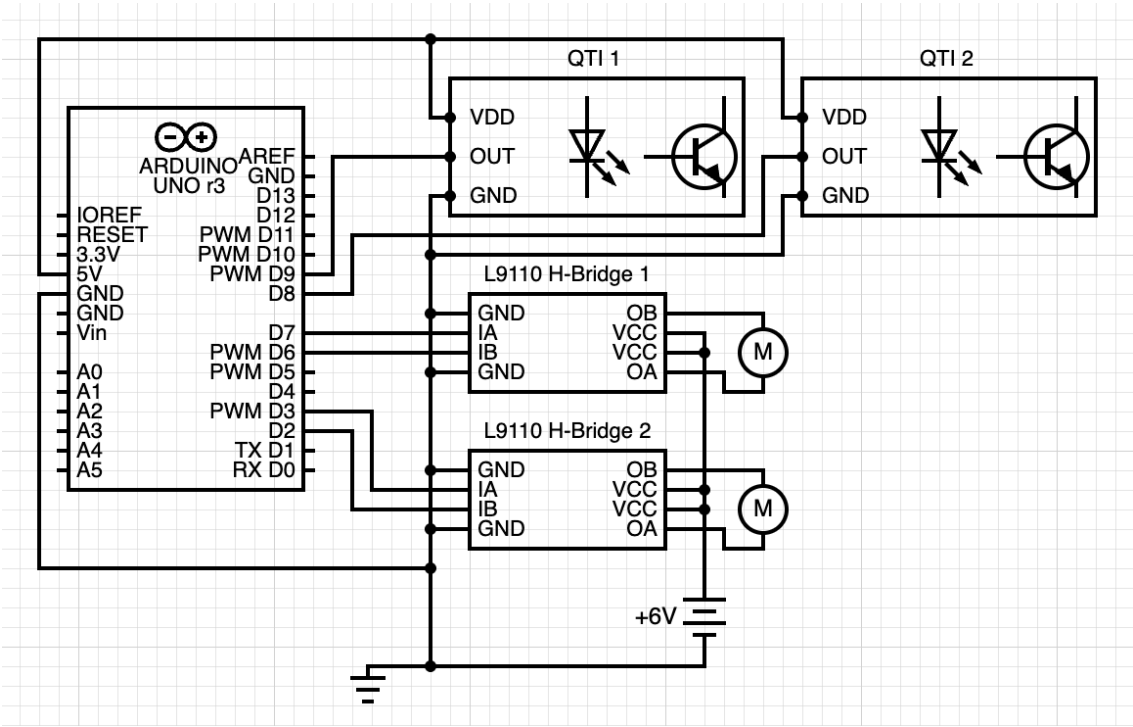
Lastly, there were some issues with the cardboard portions of the robot getting bent, which led to blocks sliding out of the perimeter of the robot. Our plan for the robot was to make the outer layer out of acrylic, but we encountered problems with the laser cutter not cutting through it. We were unable to cut our pieces manually before the competition, so we ended up using cardboard. If we were to do this project again, we would try to submit laser cutting requests earlier so that we could have pieces recut if necessary.

Our advice to future students would be to plan for the robots to hit each other often during the competition, and to start the work early so that unexpected problems can be fixed.

Appendix A: Bill of Materials (BOM)

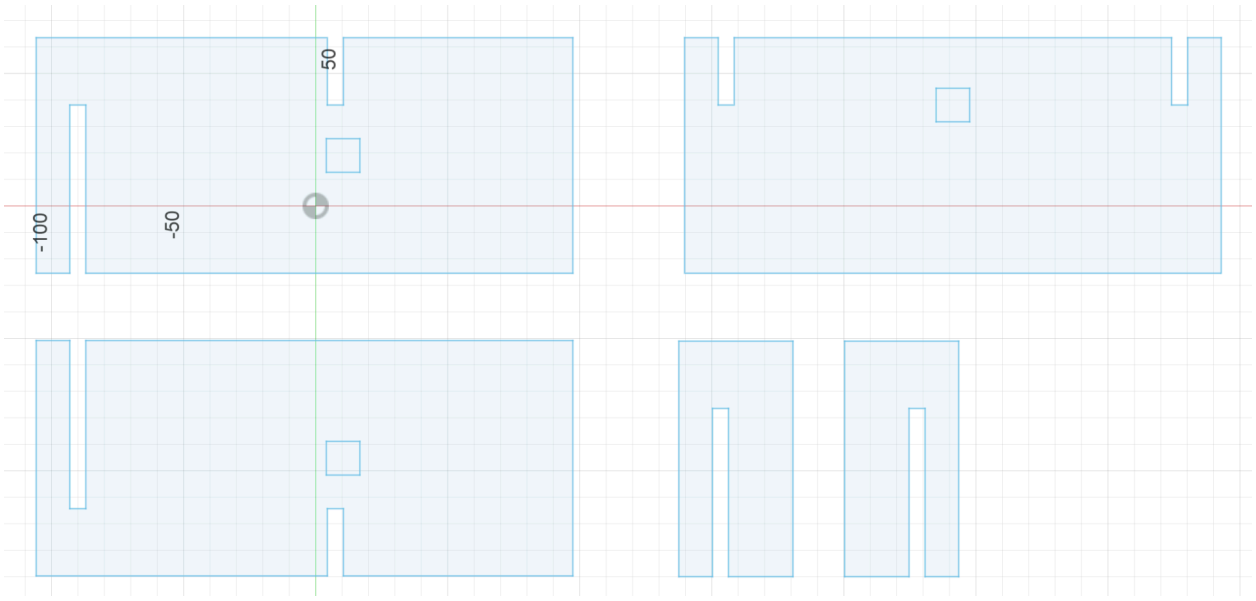
Bill of Materials				
Name	Quantity	Supplier	Cost	Notes
Laser Cut Body	1	RPL Cornell	$13.69	123.8 in cut, 5 pieces
		Project Total	$13.69	

Appendix B: Circuit Diagram



Appendix C: CAD files and drawings

Include screenshots of relevant CAD files or drawings used for manufacturing individual parts. Also include screenshots of part volumes and weight calculations (or photos of the parts being weighed) corresponding to the lines in your BOM.

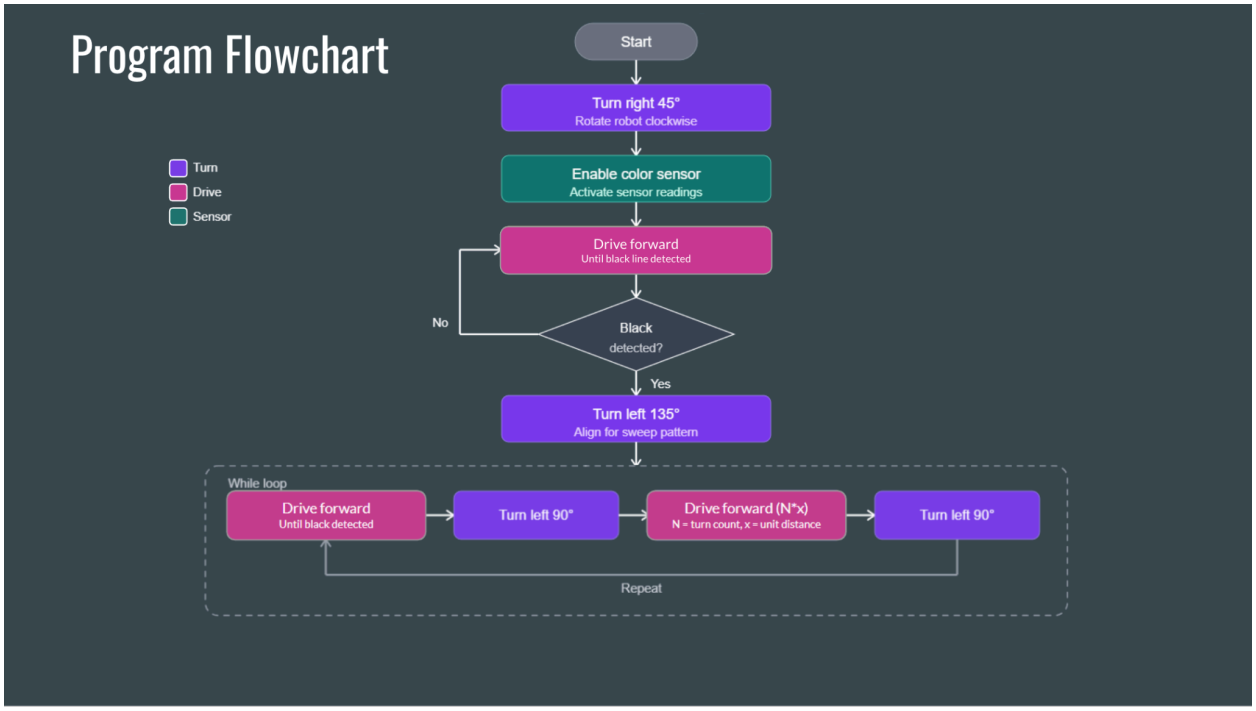


Laser Cut Cost Calculation

$0.05/inch cut + $0.50/part +$5/sheet = Total Cost

$0.05(123.8) + $0.50(5) + $5(1) = $13.69

Appendix D: Flowchart



Appendix E: Code

Commented Arduino code. A clear understanding of each line of code should be demonstrated. References must be provided if used.

```
//
================================================================================
===
// MAE 3780 — Final Robot Competition Code
// Milestone 4: Spiral Sweep
// Target hardware: Arduino Uno (ATmega328P)
// Constraint: No Arduino libraries. All hardware controlled via direct
//             register manipulation on the ATmega328P.
//
// Program overview:
//   The robot starts at the back of the playing field angled toward the
//   blue/yellow color boundary. It drives diagonally forward until it
//   crosses the color seam, then turns and executes a spiral sweep
pattern
//   to clear cubes across the entire board.
//
// Hardware connections:
```

```
//   QTI sensor 1 signal -> Arduino Pin 8  (ATmega328P port B, bit 0)
//   QTI sensor 2 signal -> Arduino Pin 9  (ATmega328P port B, bit 1)
//   Left  motor forward  -> Arduino Pin 2 (ATmega328P port D, bit 2)
//   Left  motor backward -> Arduino Pin 3 (ATmega328P port D, bit 3)
//   Right motor forward  -> Arduino Pin 6 (ATmega328P port D, bit 6)
//   Right motor backward -> Arduino Pin 7 (ATmega328P port D, bit 7)
//
// References:
//   [1] ATmega328P Datasheet, Microchip Technology, 2018.
//
https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Mic
rocontrollers-ATmega328P_Datasheet.pdf
//   [2] Parallax QTI Sensor Documentation
//       https://www.parallax.com/package/qti-sensor-documentation/
//
================================================================================
===

//
================================================================================
===
// QTI SENSOR THRESHOLDS
// These values were determined experimentally by running a debug sketch
that
// printed raw discharge counts over each surface while powered by
battery.
// A higher count means a darker surface (slower capacitor discharge).
//   Yellow surface: ~800-1000 counts
//   Blue surface:   ~1000-2700 counts
//   Black border:   ~2700+ counts
//
================================================================================
===
#define THRESHOLD_YELLOW_BLUE  1000  // counts below this = yellow, above
= blue
#define THRESHOLD_COLOR_BLACK  2700  // counts above this = black border
#define QTI_MAX_COUNT          6000  // maximum count before timing out
(prevents
                                     // infinite loop if sensor is
disconnected)
```

```
//
================================================================================
===
// TIMING CONSTANTS
// All turn times were determined experimentally by running timed turns
and
// measuring the resulting angle. Values are in milliseconds.
//
================================================================================
===
#define TURN_95_MS    800   // time to turn approximately 95 degrees left
#define TURN_90_MS    750   // time to turn approximately 90 degrees left
#define TURN_85_MS    700   // time to turn approximately 85 degrees left
#define TURN_120_MS  1150   // time to turn approximately 120 degrees left
#define STEP_MS       400   // base drive time for each spiral step
increment.
                            // Step 1 drives for 400ms, step 2 for 800ms,
etc.
#define MAX_STEPS      10   // maximum number of spiral steps before
halting.
                            // Acts as a safety limit in case the loop runs
long.

//
================================================================================
===
// CORRECTION CONSTANTS
// Used by correct_border() to nudge the robot off the black border.
//
================================================================================
===
#define CORRECTION_STEP_MS    200  // how long each nudge lasts in
milliseconds
#define MAX_CORRECTION_STEPS   21  // maximum nudges before giving up and
halting

//
================================================================================
===
```

```
// UART INITIALIZATION
// Configures the ATmega328P hardware UART for serial communication at
9600
// baud with a 16 MHz clock. Used to print debug info to the serial
monitor.
//
// Baud rate formula [1, Section 24.3.1]:
//   UBRR = (F_CPU / (16 * BAUD)) - 1 = (16,000,000 / 144,000) - 1 = 103
//   103 in binary = 0b01100111
//
================================================================================
===
void uart_init() {
  UBRR0H = 0b00000000;  // high byte of baud rate register — 0 since 103 <
256
  UBRR0L = 0b01100111;  // low byte of baud rate register — 103 = 0x67 [1]
  UCSR0B = 0b00001000;  // UCSR0B register: bit 3 (TXEN0) = 1 enables the
                        // UART transmitter. All other bits left 0. [1,
Sec 24.12.3]
  UCSR0C = 0b00000110;  // UCSR0C register: bits 2:1 (UCSZ01:UCSZ00) = 11
                        // sets 8-bit character size. Bits 4:3 = 00 sets
                        // no parity. Bit 3 = 0 sets 1 stop bit. [1, Sec
24.12.4]
}

// Sends a single character over UART.
// Polls UCSR0A bit 5 (UDRE0 — USART Data Register Empty) until the
transmit
// buffer is ready, then writes the character to UDR0. [1, Section
24.12.2]
void uart_send_char(char c) {
  while (!(UCSR0A & 0b00100000));  // wait until transmit buffer is empty
                                   // (UDRE0 is bit 5 = 0b00100000) [1]
  UDR0 = c;                        // write character to UART data
register
}

// Sends a null-terminated string over UART by calling uart_send_char for
each
```

```
// character. The pointer advances through the string until the null
terminator
// '\0' is reached, at which point *str evaluates to 0 (false) and the
loop ends.
void uart_print(const char* str) {
  while (*str) uart_send_char(*str++);  // send each char, advance pointer
}

// Sends a string followed by carriage return + newline so each message
appears
// on its own line in the serial monitor. \r\n is the standard line ending
// for serial terminals.
void uart_println(const char* str) {
  uart_print(str);          // send the string
  uart_send_char('\r');     // carriage return (moves cursor to start of
line)
  uart_send_char('\n');     // newline (moves cursor down one line)
}

// Prints an unsigned integer to the serial monitor by extracting each
digit
// using modulo 10. Digits are extracted in reverse order (least
significant
// first), stored in a buffer, then printed in forward order.
void uart_print_uint(unsigned int val) {
  if (val == 0) { uart_send_char('0'); return; }  // special case for zero
  char buf[6];   // buffer to hold up to 5 digits + null (max uint =
65535)
  int i = 0;
  while (val > 0) {
    buf[i++] = '0' + (val % 10);  // extract last digit, convert to ASCII
    val /= 10;                    // shift right one decimal place
  }
  for (int j = i - 1; j >= 0; j--) uart_send_char(buf[j]);  // print
reversed
}

//
================================================================================
===
```

```
// DELAY FUNCTIONS
// The ATmega328P runs at 16 MHz. We cannot use Arduino's delay() since we
// are not using the Arduino framework. Instead we use inline assembly
NOPs
// (No Operation instructions) for microsecond delays, and build
millisecond
// delays on top of that.
//
================================================================================
===

// Busy-wait delay in microseconds.
// Each loop iteration executes 8 NOP instructions (~8 clock cycles) plus
// loop overhead (~4 cycles) = ~12 cycles total. At 16 MHz:
//   12 cycles / 16,000,000 Hz = 0.75 µs per iteration.
// The __asm__ __volatile__ directive prevents the compiler from
optimizing
// away the NOPs since they have no visible side effects. [1, Section 6]
void delay_us(unsigned int us) {
  while (us--) {
    __asm__ __volatile__(
      "nop\nnop\nnop\nnop\n"   // 4 no-operation instructions
      "nop\nnop\nnop\nnop\n"   // 4 more no-operation instructions
    );
  }
}

// Busy-wait delay in milliseconds, built by calling delay_us 1000 times.
void delay_ms(unsigned int ms) {
  while (ms--) delay_us(1000);  // 1000 µs = 1 ms
}

//
================================================================================
===
// QTI SENSOR FUNCTIONS
// The QTI sensor works by charging a small capacitor through an IR LED
and
// phototransistor, then measuring how long the capacitor takes to
discharge.
```

```
// Dark surfaces absorb IR and discharge the capacitor slowly (high
count).
// Light surfaces reflect IR and discharge it quickly (low count). [2]
//
// To read the sensor using an Arduino digital pin:
//   1. Set the pin as OUTPUT and drive it HIGH to charge the capacitor.
//   2. Set the pin as INPUT (no pull-up) and count loop iterations until
//      the pin reads LOW (capacitor discharged).
// The count is proportional to the discharge time and therefore surface
darkness.
//
// All register operations use binary literals directly (no named
constants)
// as required by the project constraints.
//
================================================================================

// Reads QTI sensor 1 on Arduino Pin 8 (ATmega328P PB0, bit 0 of PORTB).
// DDRB controls pin direction: 1 = output, 0 = input. [1, Section 14.4]
// PORTB controls output value (or pull-up when input). [1, Section 14.4]
// PINB reads the current logic level on the pin. [1, Section 14.4]
unsigned int read_qti() {
  DDRB  |=  0b00000001;  // set PB0 as output  (DDRB bit 0 = 1)
  PORTB |=  0b00000001;  // drive PB0 HIGH to charge capacitor
  delay_us(230);         // wait 230 µs for capacitor to fully charge [2]
  DDRB  &= ~0b00000001;  // set PB0 as input   (DDRB bit 0 = 0)
  PORTB &= ~0b00000001;  // disable internal pull-up (PORTB bit 0 = 0)
                         // so capacitor discharges freely through the
sensor

  unsigned int count = 0;
  while (PINB & 0b00000001) {  // loop while PB0 reads HIGH (capacitor
discharging)
    count++;                   // increment counter each iteration
    if (count >= QTI_MAX_COUNT) break;  // timeout safety — exit if taking
too long
  }
  return count;  // higher count = darker surface
}
```

```
// Reads QTI sensor 2 on Arduino Pin 9 (ATmega328P PB1, bit 1 of PORTB).
// Identical logic to read_qti() but operates on bit 1 instead of bit 0.
unsigned int read_qti2() {
  DDRB  |=  0b00000010;  // set PB1 as output  (DDRB bit 1 = 1)
  PORTB |=  0b00000010;  // drive PB1 HIGH to charge capacitor
  delay_us(230);         // wait 230 µs for capacitor to fully charge [2]
  DDRB  &= ~0b00000010;  // set PB1 as input   (DDRB bit 1 = 0)
  PORTB &= ~0b00000010;  // disable internal pull-up

  unsigned int count = 0;
  while (PINB & 0b00000010) {  // loop while PB1 reads HIGH
    count++;
    if (count >= QTI_MAX_COUNT) break;
  }
  return count;
}

// Reads both QTI sensors and returns the higher of the two counts.
// Using the maximum means that if either sensor is over the black border,
// the combined reading will exceed THRESHOLD_COLOR_BLACK and classify as
black.
// A 2ms delay between reads prevents the two sensors from interfering
with
// each other through shared PORTB state — without it, the residual charge
// from sensor 1's cycle corrupts sensor 2's discharge timing.
unsigned int read_both_qti() {
  unsigned int a = read_qti();   // read sensor 1 (PB0)
  delay_ms(2);                   // allow PORTB to fully settle
  unsigned int b = read_qti2();  // read sensor 2 (PB1)
  return (a > b) ? a : b;        // return whichever reading is higher
}

// Classifies a QTI count value into one of three surface colors.
// Returns: 0 = yellow, 1 = blue, 2 = black
// Thresholds were determined experimentally under battery power
conditions.
uint8_t classify(unsigned int count) {
  if (count >= THRESHOLD_COLOR_BLACK) return 2;  // black border
  if (count >= THRESHOLD_YELLOW_BLUE) return 1;  // blue field
```

```
  return 0;                                          // yellow field
}

//
================================================================================
===
// MOTOR CONTROL FUNCTIONS
// Motors are driven through an H-bridge motor driver connected to PORTD.
// Each motor has a forward and backward pin; driving one HIGH and the
other
// LOW spins the motor in the corresponding direction.
//
// PORTD pin assignments (DDRD = 0b11001100):
//   PD2 (pin 2) — left  motor forward
//   PD3 (pin 3) — left  motor backward
//   PD6 (pin 6) — right motor forward
//   PD7 (pin 7) — right motor backward
//   PD0, PD1 left as inputs (used by UART TX/RX)
//   PD4, PD5 not used
//
================================================================================
===

// Configures motor pins as outputs and sets all to LOW (motors off).
// DDRD bit = 1 sets that pin as output. [1, Section 14.4]
void motors_init() {
  DDRD  = 0b11001100;  // set PD7, PD6, PD3, PD2 as outputs; others as
inputs
  PORTD = 0b00000000;  // initialize all motor pins LOW (motors stopped)
}

// Stops all motors by driving all motor pins LOW.
// ANDing with 0b00000000 clears all bits in PORTD.
void stop_motors() {
  PORTD &= 0b00000000;  // all motor pins LOW = no current through motors
}

// Drives both motors forward.
// Left forward  (PD2) = HIGH, Left backward  (PD3) = LOW
// Right forward (PD6) = HIGH, Right backward (PD7) = LOW
```

```
void drive_forward() {
  PORTD |=  0b10000100;  // set PD7=1 ... wait, set PD6=1 (right fwd), PD2=1 (left fwd)
  PORTD &=  0b10111101;  // clear PD3 (left bwd) and PD6... ensure opposing pins LOW
}

// Drives both motors in reverse.
// Left backward  (PD3) = HIGH, Left forward  (PD2) = LOW
// Right backward (PD7) = HIGH, Right forward (PD6) = LOW
void drive_backward() {
  PORTD |=  0b01001000;  // set PD6=1 (right bwd), PD3=1 (left bwd)
  PORTD &=  0b01110111;  // clear opposing forward pins
}

// Spins the robot left in place.
// Left motor backward (PD3 HIGH), Right motor forward (PD6 HIGH).
// Opposing pins driven LOW to prevent shoot-through in H-bridge.
void turn_left() {
  PORTD |=  0b10001000;  // PD7=1... PD6=1 (right fwd), PD3=1 (left bwd)
  PORTD &=  0b10111011;  // clear PD6... clear opposing pins
}

// Spins the robot right in place.
// Left motor forward (PD2 HIGH), Right motor backward (PD7 HIGH).
void turn_right() {
  PORTD |=  0b01000100;  // PD6=1 (right bwd), PD2=1 (left fwd)
  PORTD &=  0b01110111;  // clear opposing pins
}

// Turns left for a specified duration in milliseconds, then stops.
// A 100ms settle delay after stopping prevents momentum from carrying
// the robot further than intended.
void turn_left_ms(unsigned int ms) {
  turn_left();       // begin spinning left
  delay_ms(ms);      // hold for specified duration
  stop_motors();     // cut power to motors
  delay_ms(100);     // wait for robot to physically stop before next action
}
```

```
//
================================================================================
===
// BORDER CORRECTION
// Called when the robot unexpectedly hits the black border mid-drive.
// Nudges the robot in small steps to get back onto a colored surface.
// Strategy: try turning left for the first third of MAX_CORRECTION_STEPS,
// then switch to turning right. This handles robots hitting the border at
// various angles — one direction will always steer back to the field.
//
================================================================================
===
void correct_border() {
  uart_println("Correcting border...");
  stop_motors();    // stop immediately on border detection
  delay_ms(50);     // brief pause before beginning correction

  uint8_t steps = 0;
  uint8_t third = MAX_CORRECTION_STEPS / 3;  // = 7 steps left, then
switch right

  while (steps < MAX_CORRECTION_STEPS) {
    if (steps < third) {
      turn_left();   // first 7 steps: nudge left
    } else {
      turn_right();  // remaining steps: nudge right
    }
    delay_ms(CORRECTION_STEP_MS);  // nudge for 200ms per step
    stop_motors();                 // stop between nudges to take a clean
reading

    unsigned int count = read_both_qti();   // read both sensors
    uint8_t surface = classify(count);      // classify the surface
    if (surface != 2) {                     // if no longer on black,
correction done
      uart_println("Correction done.");
      stop_motors();
      delay_ms(50);
      return;  // return to calling function to resume normal behavior
```

```
    }
    steps++;  // still on black — try next nudge
  }

  // If we reach here, all correction attempts failed — halt for safety
  uart_println("Correction failed! Halting.");
  stop_motors();
  while (1);  // infinite loop — requires manual reset
}

//
================================================================================
===
// DRIVE UNTIL BLACK
// Drives the robot forward until either QTI sensor detects the black
border.
// Uses a stop-read-restart pattern on each iteration: motors are stopped
for
// 5ms before each QTI read to allow the power rail voltage to stabilize.
// When motors run, they draw large currents that cause voltage
fluctuations
// which corrupt the QTI capacitor discharge timing, causing
misclassification.
// Stopping briefly before each read eliminates this noise source.
//
================================================================================
===
void drive_until_black() {
  drive_forward();  // begin driving
  while (1) {
    delay_ms(50);       // drive for 50ms before checking (allows
meaningful movement)
    stop_motors();      // stop motors to eliminate electrical noise on
power rail
    delay_ms(5);        // wait 5ms for voltage to stabilize
    unsigned int count = read_both_qti();   // read both sensors
    uint8_t surface = classify(count);      // classify surface

    if (surface == 2) {  // black border detected
      stop_motors();
```

```
      uart_print("Border hit. count=");
      uart_print_uint(count);   // print raw count for debugging
      uart_println("");
      return;  // return to caller — robot is stopped at border
    }

    drive_forward();  // not black yet — resume driving
  }
}

//
================================================================================
===
// DRIVE TIMED
// Drives the robot forward for a fixed duration in milliseconds.
// Used for the sideways steps in the spiral sweep where distance is
controlled
// by time rather than sensor feedback.
//
================================================================================
===
void drive_timed(unsigned int ms) {
  drive_forward();   // begin driving
  delay_ms(ms);      // drive for specified duration
  stop_motors();     // stop
  delay_ms(50);      // brief settle before next action
}

//
================================================================================
===
// MAIN
// Entry point. Executes the full competition sequence:
//   1. Detect starting color (yellow or blue)
//   2. Drive diagonally to the color seam
//   3. Turn to face across the board
//   4. Execute spiral sweep until MAX_STEPS reached
//
================================================================================
===
```

```
int main() {
  sei();           // enable global interrupts (required for UART to
function
                   // correctly on the ATmega328P) [1, Section 7.3]
  uart_init();     // configure UART for serial debug output
  motors_init();   // configure motor pins as outputs

  uart_println("=== Milestone 4: Spiral Sweep ===");
  delay_ms(500);  // 500ms power-on settle time before reading sensors

  // --- Step 1: Detect starting color ---
  // The robot can be placed on either yellow or blue. We read the sensor
  // now so we know which color change to look for when finding the seam.
  uart_println("Reading starting color...");
  unsigned int start_count = read_both_qti();     // take initial sensor
reading
  uint8_t start_color = classify(start_count);    // classify as yellow(0)
or blue(1)

  if (start_color == 2) {  // started on black border — should not happen
    uart_println("Started on black! Halting.");
    stop_motors();
    while (1);  // halt and wait for manual reset
  }

  uart_print("Starting color: ");
  uart_println(start_color == 0 ? "YELLOW" : "BLUE");  // print color for
debug

  // --- Step 2: Drive diagonally to the color seam ---
  // Drive forward until the sensor detects a color different from
start_color.
  // If the black border is hit, call correct_border() to recover, then
resume.
  // Track whether any correction occurred — it affects the turn angle in
Step 3.
  uart_println("Driving to color seam...");
  uint8_t correction_happened = 0;  // flag: 0 = no correction, 1 =
corrected
  drive_forward();
```

```
  while (1) {
    delay_ms(50);     // drive 50ms between reads
    stop_motors();    // stop for clean sensor read
    delay_ms(5);      // let voltage stabilize
    unsigned int count = read_both_qti();
    uint8_t surface = classify(count);

    if (surface == 2) {           // hit black border unexpectedly
      correction_happened = 1;    // record that correction occurred
      uart_println("Border hit, correcting...");
      correct_border();           // nudge back onto colored surface
      drive_forward();            // resume driving toward seam
    } else if (surface != start_color) {  // color changed — seam reached
      stop_motors();
      uart_println("Seam reached. Stopping.");
      break;  // exit loop and proceed to Step 3
    } else {
      drive_forward();  // still on start color — keep driving
    }
  }

  delay_ms(200);  // brief pause at seam before turning

  // --- Step 3: Turn to face across the board ---
  // The required turn angle depends on whether border correction
occurred.
  // Without correction: robot arrived at seam at ~60° to the board edge,
  //   so a 120° left turn aligns it perpendicular to the board.
  // With correction: correct_border() left the robot perpendicular to the
  //   border, so only a 90° turn is needed to face across the board.
  if (correction_happened) {
    uart_println("Correction occurred - turning left 90 degrees...");
    turn_left_ms(TURN_90_MS);   // 90 degree turn (750ms)
  } else {
    uart_println("No correction - turning left 120 degrees...");
    turn_left_ms(TURN_120_MS);  // 120 degree turn (1150ms)
  }
  uart_println("Aligned. Beginning spiral sweep.");

  // --- Step 4: Spiral sweep loop ---
```

```
  // Each iteration of this loop performs one full sweep-and-step cycle:
  //   a. Drive straight across the board until hitting the black border
  //   b. Turn left ~95 degrees
  //   c. Drive sideways for (step * STEP_MS) milliseconds
  //      — step 1 = 400ms, step 2 = 800ms, step 3 = 1200ms, etc.
  //      — increasing step size offsets each cross-drive further along
the board
  //   d. Turn left ~85 degrees to face back across the board
  //   Repeat with step incremented, creating a progressive spiral pattern
  //   that sweeps the entire playing field.

  uint8_t step = 1;  // step counter — controls sideways drive distance

  while (step <= MAX_STEPS) {
    uart_print("Spiral step ");
    uart_print_uint(step);
    uart_println(":");

    // a. Drive across the board until hitting the black border
    uart_println("  Driving to border...");
    drive_until_black();  // sensor-guided drive — stops when black
detected

    // b. Turn left ~95 degrees to face along the board
    // 95 degrees used (rather than 90) to compensate for robot drift
    uart_println("  Turning left 95...");
    turn_left_ms(TURN_95_MS);  // TURN_95_MS = 800ms

    // c. Drive sideways for (step * STEP_MS) milliseconds
    // Each successive step is one increment wider to advance the sweep
position
    unsigned int side_ms = step * STEP_MS;  // e.g. step 3 → 3 * 400 =
1200ms
    uart_print("  Driving sideways for ");
    uart_print_uint(side_ms);
    uart_println("ms...");
    drive_timed(side_ms);  // timed drive — no sensor feedback needed here

    // d. Turn left ~85 degrees to face back across the board
    // 85 degrees used (rather than 90) to compensate for robot drift;
```

```
    // the 95 + 85 = 180 total ensures the robot faces the opposite
direction
    uart_println("  Turning left 85...");
    turn_left_ms(TURN_85_MS);  // TURN_85_MS = 700ms

    step++;  // increment step counter for next iteration
  }

  // All spiral steps complete — stop and hold
  uart_println("Spiral sweep complete!");
  stop_motors();
  while (1);  // hold here — competition run is finished
  return 0;
}
```